

NBA Pronos - Donnees stats

Dossier Cadrage/Stats/ - mis a jour le 21/08/2026

Sommaire

1. Objectif
2. Ou se trouvent les fichiers
3. Portee des donnees extraites
4. Lancer l'extraction (fetch_nba_data.py)
5. Mettre a jour la base (load_to_sqlite.py)
6. Si 'python' n'est pas reconnu
7. Comprendre la base de donnees
8. Construire les features et les cibles
9. Entraîner les modeles
10. Comprendre les modeles sauvegardes
11. Tester un modele en ligne de commande
12. Service de prediction en production (Cloud Run)
13. Rafraichissement quotidien automatique (Supabase)
14. Paris personnalisés pilotes par l'IA (Phase 5)
15. Suivre l'etat du projet
16. Prochaines etapes
17. Enchainements courants (que faire selon l'objectif)

1. Objectif

Recuperer les feuilles de match NBA (box scores + play-by-play) des deux dernieres saisons, les structurer dans une base de donnees, puis entrainer des modeles pour estimer des probabilites d'evenements (points d'un joueur, double-double, pourcentage de tir...). Le detail du cadrage, des decisions et des resultats est dans projet-data-nba.md, dans ce meme dossier -- notamment le bandeau REPRISE en tete de ce fichier, toujours a jour en fin de session.

2. Ou se trouvent les fichiers

```
Cadrage/Stats/
projet-data-nba.md      <- cadrage d'origine + journal des decisions
README.pdf             <- ce document
Dockerfile, .dockerignore <- image du service Cloud Run (S12)
scripts/
  fetch_nba_data.py     <- extraction (nba_api -> CSV)
  load_to_sqlite.py     <- chargement (CSV -> base SQLite)
  build_features.py     <- variables predictives (features_*)
  build_targets.py      <- tables cibles (labels_*, entraînement_*)
  train_points_model.py <- modele des points (reference)
```

```

train_doubledouble_model.py <- double-double / triple-double
train_stat_model.py         <- reb/ast/fg3m/stl/blk/minutes (6 modeles)
train_pct_model.py          <- FT%/FG%/3P% (3 modeles, approche taux)
demo_pari_reel.py           <- demo cablee en dur (Tatum, points)
tester_modele.py            <- teste N'IMPORTE quel modele en ligne de commande
calibrate_difficulty_thresholds.py <- calibre les seuils proba->palier (S16/S17)
generate_readme.py          <- regenere ce document
requirements.txt

service/                     <- micro-service FastAPI, deploye sur Cloud Run (S12)
  app.py                     <- endpoints /health, /predict, /refresh
  supabase_context.py        <- equivalent Postgres de build_context()/compute_proba()
  refresh_daily.py           <- fetch incremental quotidien -> Supabase (S13)
  backfill_supabase.py       <- migration ponctuelle nba.db -> Supabase
  DEPLOIEMENT_CLOUD_RUN.md  <- guide de deploiement pas-a-pas
data/                        <- NON versionne dans git (trop volumineux)
  raw/2024-25/, raw/2025-26/ <- CSV bruts (games_index, boxscores...)
  logs/                      <- journaux d'execution de l'extraction
  nba.db                     <- la base de donnees SQLite (usage LOCAL/dev)
models/                      <- NON versionne dans git
  *.joblib                   <- modeles entraines et sauvegardes (S9)

```

data/ et models/ sont exclus de git (.gitignore) : volumineux et entierement regenerables en relancant les scripts (S4-S9). En PRODUCTION (service Cloud Run), la source de donnees n'est plus nba.db mais les tables Supabase stats_equipes/stats_joueurs/stats_box_scores/ stats_matches (S12/S13) -- nba.db reste utile en local pour entrainer et tester les modeles.

3. Portee des donnees extraites

Parametre	Valeur retenue
Saisons	2021-22 a 2025-26 (5 saisons)
Types de match	Saison reguliere, Playoffs, Play-In Tournament
Box score	Traditionnel (pts/reb/ast...) + Avance (ratings, usage%, pace...)
Play-by-play	Detail de chaque action du match (tirs, fautes, remplacements...)
Volume	6 602 matchs (extraction terminee, 0 echec), ~3 fichiers par match

4. Lancer l'extraction (fetch_nba_data.py)

Ce script va chercher les données sur l'API NBA (nba_api) et les enregistrer en CSV dans data/raw/. Il est resumable : s'il est interrompu (fenêtre fermée, PC éteint, coupure réseau), il suffit de relancer la même commande -- il saute automatiquement les matchs déjà téléchargés.

Ouvrir un terminal PowerShell, se placer dans le dossier des scripts :

```
cd C:\dev\nba-pronos\Cadrage\Stats\scripts
```

Puis lancer :

```
python fetch_nba_data.py
```

Extraction des 5 saisons (2021-22 à 2025-26) déjà TERMINÉE au 20/08/2026 (6 602/6 602 matchs, 0 échec) -- cette section reste utile pour une saison future ou un complément. Durée estimée pour un fetch complet : plusieurs heures (pause volontaire entre chaque appel à l'API pour éviter un blocage par NBA.com).

Pour que le script continue de tourner même si VS Code est fermé, le lancer dans une fenêtre détachée :

```
Start-Process python -ArgumentList "fetch_nba_data.py" \
-WorkingDirectory "C:\dev\nba-pronos\Cadrage\Stats\scripts"
```

Une nouvelle fenêtre de console s'ouvre : ne pas la fermer, elle peut être minimisée.

5. Mettre à jour la base (load_to_sqlite.py)

Ce script relit tous les CSV présents dans data/raw/ et reconstruit entièrement la base data/nba.db. Durée variable selon le volume : de quelques secondes (2 saisons) à plusieurs minutes (5 saisons et plus, ~19 000 fichiers) -- affiche l'avancement en continu (saison par saison en lecture, table par table en écriture), pas besoin d'attendre la fin pour savoir si ça tourne. Sans risque : peut être relancé autant de fois que nécessaire, y compris pendant que l'extraction tourne encore.

```
python load_to_sqlite.py
```

Un résumé s'affiche à la fin (nombre de lignes par table).

ATTENTION si interrompu (Ctrl+C) en cours de lecture : les tables sont vidées en tout début d'exécution, l'écriture se fait en un seul bloc à la fin -- une interruption avant "Écriture dans la base..." laisse la base VIDE jusqu'à la prochaine exécution COMPLETE (relancer jusqu'au bout, pas de reprise partielle possible ici). Ne JAMAIS lancer 2 instances en même temps volontairement (2 lancements accidentels restent tolérables : timeout de 120s sur le verrou SQLite, l'un des deux attend l'autre plutôt que de planter).

6. Si 'python' n'est pas reconnu

Utiliser le chemin complet à la place, pour n'importe quel script de ce dossier :

```
C:\Users\lenoi\AppData\Local\Programs\Python\Python312\python.exe fetch_nba_data.py
C:\Users\lenoi\AppData\Local\Programs\Python\Python312\python.exe load_to_sqlite.py
```

Cela arrive si le terminal a ete ouvert avant l'installation de Python sur la machine. Fermer et rouvrir VS Code regle generalement le probleme.

7. Comprendre la base de donnees

La base contient les memes informations que les CSV, mais organisees en tableaux relies entre eux plutot qu'eparpillees dans des milliers de fichiers -- ce qui permet de poser des questions directement (ex : "tous les matchs de Jokic cette saison").

Tableau	Contenu
equipes	Les 30 equipes NBA (une ligne chacune)
joueurs	Tous les joueurs apparus dans les matchs extraits
matchs	Un match par ligne : date, saison, equipes, score final
box_scores	Stats d'un joueur sur un match donne (pts, rebonds, passes...)
box_scores_advanced	Idem, stats avancees (rating offensif, usage%, pace...)
play_by_play	Chaque action du match dans l'ordre chronologique
features_equipe	1 ligne/(match, equipe) : variables predictives (S8)
features_joueur	1 ligne/(match, joueur) : variables predictives (S8)
labels_joueur	1 ligne/(match, joueur) : resultats reels a predire (S8)
entrainement_matches	1 ligne/match : features + resultat (victoire/ecart) (S8)

Les 6 premiers tableaux sont relies par des identifiants communs (game_id, player_id, team_id). Les 4 derniers (prefixe features_/labels_/entrainement_) sont DERIVES des precedents par build_features.py/build_targets.py (S8) -- reconstruits entierement a chaque execution, jamais edites a la main.

8. Construire les features et les cibles

Avant d'entrainer un modele, 2 scripts transforment les tableaux bruts (box_scores, matchs...) en variables predictives et en resultats reels a predire :

- build_features.py -- calcule les moyennes glissantes (5/10 derniers matchs), le repos, le contexte...
Regle anti-fuite stricte : chaque variable pour un match donne n'utilise QUE l'historique strictement anterieur (jamais le resultat du match qu'elle sert a predire).
- build_targets.py -- construit les tables labels_joueur
(pts/reb/ast/3-points/interceptions/contres/minutes/lancers-francs/tirs REELS, un match a la fois) et entrainement_matches (resultat de chaque match).

A lancer, dans cet ordre, apres toute mise a jour de la base (S5) :

```
python build_features.py
python build_targets.py
```

Rapide (quelques secondes), reconstruit entierement les tables a chaque execution -- sans risque, a relancer aussi souvent que necessaire.

9. Entraîner les modeles

Une fois les features et les cibles construites (S8), 4 scripts entraînent et sauvegardent les modeles (dans models/*.joblib) :

Script	Modele(s) produit(s)
train_points_model.py	points.joblib
train_doubledouble_model.py	double_double.joblib, triple_double.joblib
train_stat_model.py	reb, ast, fg3m, stl, blk, min .joblib (6 modeles)
train_pct_model.py	ft_pct, fg_pct, fg3_pct .joblib (3 modeles)

Se placer dans le dossier des scripts, puis lancer chaque script un a un :

```
cd C:\dev\nba-pronos\Cadrage\Stats\scripts
python train_points_model.py
python train_doubledouble_model.py
python train_stat_model.py
python train_pct_model.py
```

Duree : quelques secondes a 1-2 minutes par script (RandomForest sur ~56 000 lignes) -- rien a voir avec les plusieurs heures de l'extraction (S4). Chaque script ECRASE le(s) fichier(s) .joblib existant(s) : relancer un script apres une mise a jour de la base (S5-S8) reentraîne sur les donnees les plus recentes, aucune etape manuelle supplementaire.

Chaque script affiche a l'ecran ses propres metriques de calibration (MAE, R^2 , un tableau "seuil | proba moyenne predite | taux reel | ecart") -- c'est la meme lecture a chaque fois : plus l'ecart est proche de 0%, mieux le modele est calibre. Rien a enregistrer a la main, la sortie console suffit pour verifier qu'un changement n'a pas degrade un modele.

10. Comprendre les modeles sauvegardes

Chaque fichier models/*.joblib contient un dictionnaire Python (charge avec `joblib.load(...)`) : le modele scikit-learn entraine, la liste des colonnes de features qu'il attend, et les parametres necessaires pour calculer une probabilite SANS reentraîner (ecart-type pour une distribution normale, ou parametres de la loi Binomiale pour un pourcentage de tir -- voir projet-data-nba.md S18).

Exemple de chargement, dans un script ou une console Python :

```
import joblib
m = joblib.load('../models/points.joblib')
print(m['target'], m['distribution'])
```

11. Tester un modele en ligne de commande

tester_modele.py fonctionne pour les 12 modeles a la fois -- donne un joueur, une stat et un seuil, il choisit lui-meme le bon fichier .joblib et la bonne formule (regression+distribution, classification directe, ou Binomial pour un pourcentage de tir). Reconstitue le contexte du PROCHAIN match a partir des 10 derniers matchs REELEMENT connus du joueur.

```
python tester_modele.py --joueur "Nikola Jokic" --stat pts --seuil 25  
python tester_modele.py --joueur "Jokic" --stat dd  
python tester_modele.py --joueur "Tatum" --stat ft --seuil 0.85
```

Code --stat	Modele(s)
pts, reb, ast, fg3m, stl, blk, min	regression + distribution (--seuil obligatoire)
dd, td	double-double / triple-double (probabilite directe)
ft, fg, fg3	% de tir -- --seuil en fraction (0.80 = 80%)

--adversaire (code/ville/nom d'equipe) n'affecte que --stat pts, seul modele a utiliser l'historique face a un adversaire precis. --exterieur/--repos ajustent le contexte du prochain match (illustratif si non precise). Recherche de joueur/equipe insensible aux accents (un nom tape sans accent trouve le nom accentue en base).
demo_pari_reel.py reste le 1er exemple historique (un seul cas cable en dur, points uniquement) --
tester_modele.py le generalise aux 12 modeles.

12. Service de prediction en production (Cloud Run)

Le moteur de proba (les 12 modeles + `compute_proba()`) est expose en production via un micro-service FastAPI (`service/app.py`), deploye sur Google Cloud Run (conteneur sans serveur, `scale-to-zero` -- pas de cout quand personne ne l'appelle). Il lit les stats joueurs/equipes/matches depuis Supabase (`service/supabase_context.py`, cle `service_role`) plutot que depuis `nba.db` -- `nba.db` reste local, utilise seulement pour entrainer/tester les modeles (S9/S11).

Endpoint	Role
GET /health	verification que le service tourne
POST /predict	joueur + stat + seuil -> proba (appele par l'appli Next.js)
POST /refresh	stub reserve, le vrai refresh est S13 (job separe)

L'appli Next.js appelle `/predict` depuis `lib/ai/statsService.ts` (`predictOverUnder()`), a la structuration d'un pari personnalise (S14).

Guide de deployment complet, pas-a-pas (gcloud, secrets, IAM) : `service/DEPLOIEMENT_CLOUD_RUN.md`.

Redeployer apres toute modification de `service/app.py`, `service/supabase_context.py` ou `scripts/tester_modele.py` :

```
gcloud run deploy nba-pronos-stats --source . --region europe-west1
```

(depuis `Cadrage/Stats/`, le `Dockerfile` ne copie que `scripts/tester_modele.py`, `models/`, `service/app.py` et `service/supabase_context.py` -- jamais `data/`).

13. Rafraichissement quotidien automatique (Supabase)

`service/refresh_daily.py` va chercher les nouveaux matchs joues depuis la derniere execution (`nba_api`) et les ajoute aux tables Supabase -- sans reduplication ni re-telechargement des matchs deja connus (`known_game_ids()`, pagine correctement au-dela de 1000 lignes). Programme pour tourner automatiquement chaque jour via un workflow GitHub Actions (`.github/workflows/`), pour que les stats utilisees par `/predict` restent a jour sans intervention manuelle.

En cas de probleme (job en echec, donnees manquantes) : verifier d'abord les logs du workflow dans l'onglet Actions du depot GitHub.

14. Paris personnalisés pilotes par l'IA (Phase 5)

Quand un joueur soumet un pari personnalise en texte libre (ex. "MPJ marque plus de 25 points"), l'appli le fait analyser par l'IA (`Claude`, `lib/ai/structureBet.ts`) qui en extrait le joueur, la stat, le seuil et le sens (plus/moins de), en verifiant que le joueur fait bien partie d'une des deux equipes du match vise (contexte transmis a l'IA) et en corrigeant les fautes d'orthographe grace a sa connaissance NBA.

- Si le pari est calculable, la proba est calculee via le service Cloud Run (S12) et le pari est AUTO-VALIDE immediatement (plus de queue d'admin a traiter pour ces cas) -- la proba n'est montree au joueur qu'une fois le pari valide, jamais avant.
- Si le joueur cite est un vrai joueur NBA mais ne joue pas dans CE match precis, le pari est quand meme accepte (jamais bloquant) mais avec une proba forcee a 0% -- il perdra simplement a la resolution.
- Si le pari n'est pas calculable (evenement hors-terrain, stat non supportee...), le mecanisme manuel existant prend le relais : difficulte choisie a la main par un admin, comme avant l'IA.
- Un admin peut a tout moment corriger la difficulte d'un pari auto-valide par l'IA (section dediee

"Auto-valides par l'IA" de la page d'administration des paris).

Detail complet de la conception et des decisions :

Cadrage/V1/SPEC_TECHNIQUE_PROBA_PARIS_PERSOS_V0_1.md (S7bis). Limite connue : la verification d'equipe s'appuie sur la connaissance NBA de l'IA, pas sur un vrai roster a jour en temps reel.

15. Suivre l'etat du projet

- Demander directement dans la conversation : je verifie et je reponds.
- Lire le bandeau REPRISE en tete de projet-data-nba.md -- toujours a jour en fin de session, resume l'etat et la prochaine etape a trancher.
- Journal complet des sessions et decisions : Cadrage/Proto/JOURNAL_SESSIONS.md (section "Projet Data NBA").

16. Prochaines etapes

- Extraction ETENDUE a 5 saisons TERMINEE (6 602/6 602 matchs, 0 echec) -- plus a relancer sauf pour une saison future.
- Phase 1 (12 modeles) et Phase 2 (reentrainement sur les 5 saisons completes) TERMINEES.
- Phase 4 (service Cloud Run + rafraichissement quotidien Supabase, S12/S13) et Phase 5 (paris IA, S14) EN PRODUCTION.
- Calibration reelle des seuils proba->difficulte (SPEC_TECHNIQUE_PROBA_PARIS_PERSOS_V0_1.md S7 point 2) : en cours via `calibrate_difficulty_thresholds.py`, qui simule une large distribution de probas plutot que d'attendre un vrai volume de paris joueurs.
- Reste a trancher (voir GAPS_OUVERTS.md) : bareme de repli pour les paris non calculables par l'IA (point 5 du meme S7), assistance IA a la RESOLUTION des paris une fois un match termine (idee notee, pas encore construite), repli automatique des cartes apres soumission d'un pari/prono (piste UX, pas encore construite).

17. Enchainements courants (que faire selon l'objectif)

Recapitulatif des commandes des sections precedentes, regroupees par objectif plutot que par script -- pour trouver directement quoi lancer sans reparcourir tout le document.

- Verifier vite un pari sur un joueur : `tester_modele.py --joueur ... --stat ... --seuil ...` (S11).
- Ajouter une nouvelle saison / rattraper du retard local : dans l'ordre, `fetch_nba_data.py -> load_to_sqlite.py -> build_features.py -> build_targets.py -> train_points_model.py -> train_doubledouble_model.py -> train_stat_model.py -> train_pct_model.py` (S4-S9).
- Faire beneficier l'app en prod des derniers matchs joues : rien a faire, `refresh_daily.py` tourne seul chaque jour a 10h (GitHub Actions) -- en cas de doute, verifier l'onglet Actions du depot (S13).
- Reentraîner les modeles sur les donnees les plus recentes : `build_features.py -> build_targets.py -> les 4 train_*.py` (S8/S9), PUIS redeployer (point suivant) pour que la prod utilise les nouveaux .joblib.
- Redeployer apres une modification de code du service : depuis Cadrage/Stats/, `gcloud run deploy nba-pronos-stats --source . --region europe-west1` -- necessaire des que `app.py`, `supabase_context.py` ou `tester_modele.py` changent (S12).
- Recalibrer les seuils proba -> palier de difficulte : `calibrate_difficulty_thresholds.py`, puis reporter les seuils affiches dans `lib/ai/difficultyTiers.ts` (cote app Next.js, pas ce dossier).
- Reinitialiser Supabase depuis zero (rare, deja fait une fois) : `backfill_supabase.py -- ECRASE` entierement les tables `stats_*` (S12/S13).
- 'python' non reconnu dans le terminal : utiliser le chemin complet vers `python.exe`, cf S6.